

EJERCICIO 1) 10 PUNTOS

Algunas de las rutas de nuestro país están deterioradas, tienen pozos o roturas. Por esta razón a cada tramo de ruta entre distintas ciudades se le asignó un grado de “mal estado”, mientras más alto, peor está el tramo de ruta y más daño hace a los vehículos.

Se requiere implementar un algoritmo que dado un par de ciudades origen y destino, devuelva el mejor trayecto de ciudades para llegar de una a la otra. Definimos mejor trayecto a aquel que daña menos los vehículos, o sea, que tiene menor la suma total de grados de mal estado.

- a) Asumiendo que las ciudades son identificadas por naturales y el grado de los tramos de rutas también son naturales, explique cómo modelaría el problema y cómo cargaría los datos al modelo.
- b) Implementar la siguiente función que devuelve una tupla con un iterador y un natural. El iterador contiene a los identificadores de las ciudades que pertenecen al mejor trayecto entre la ciudad origen y destino. El natural indica el grado de mal estado total del trayecto. Se debe implementar en $O(C^2)$ siendo C la cantidad máxima de ciudades.

Tupla<Iterador<nat>,nat> mejorTrayecto(nat ciudadOrigen, nat ciudadDestino)

- c) A su vez, se requiere construir un registro que indique para cualquier par de ciudades, cuál es el grado de mal estado del mejor trayecto entre ellas. Explique por lo menos dos formas de resolver el problema.
- d) Implemente en la siguiente función uno de los algoritmos anteriores. Debe retornar una matriz que para cada ciudad i, j , $mat[i][j]$ indica el grado total de mal estado del trayecto entre dichas ciudades.

Matriz<nat> gradosDeTrayectos()

En caso de utilizar TADs auxiliares deberá especificarlos, pero no implementarlos.

Duración: 3 horas

Con material: NO

EJERCICIO 2) 10 PUNTOS

Un estudiante muy organizado decide armar un sistema para tener una mejor organización de las materias y de los eventos de la universidad. Las materias son reconocidas por un código y se quiere guardar su nombre, el nombre del profesor y la nota de aprobación (1-100), junto con los eventos que tiene. De los eventos se reconoce la fecha, el nombre, y una prioridad para ser atendido, la cual va a ser calculada a mano antes de ingresarse al sistema.

Se sabe que no va a haber mas de MAX_MATERIAS y MAX_EVENTOS en el sistema.

Se requiere poder implementar las siguientes operaciones:

1) Agregar Materia

void AgregarMateria (Estructura e, Cadena codigo, Cadena nombre, Cadena profesor)

Esta operación permite el ingreso al sistema de una nueva materia identificada por el código al sistema. Esta operación debe cumplirse en $O(\ln n)$ peor caso. Siendo n la cantidad de materias registradas en el sistema

2) Agregar Evento Pendiente

void AgregarEvento (Estructura e, Cadena codigoM, Cadena nombre, Cadena fecha, nat prioridad)

Esta operación permite el ingreso al sistema de un nuevo evento identificado por su nombre, relacionado con la materia cuyo código es pasado por parámetro. Esta operación debe cumplirse en $O(\ln n)$ peor caso, siendo n la cantidad de eventos registrados en el sistema.

3) Listado Materias

void ListadoMateriasYEventos (Estructura e)

Duración: 3 horas

Con material: NO

Escuela de Ingeniería

Parcial 1 de: Estructuras de Datos y Algoritmos 2

Código de materia: 1778

Fecha: 16/05/2018

Nocturno

Hoja 3 de 3

Esta operación imprime por pantalla las materias ordenadas por nombre, y para cada uno de las materias los eventos relacionados. Esta operación debe cumplirse en $O(n)$, siendo n la cantidad de materias registradas en el sistema.

4) Evento A Preparar

void EventoAPreparar (Estructura e)

Esta operación imprime por pantalla el evento con mas urgencia a ser preparado. Esta operación debe cumplirse en $O(1)$ peor caso.

5) Eliminar Evento de los Pendientes

void EliminarEventoPendientes (Estructura e)

Esta operación elimina el evento con mas prioridad del sistema, de forma que se pueda saber cual es el proximo a ser atendido. Esta operación debe cumplirse en $O(\ln n)$ peor caso.

6) Materias por Nota

void MateriasPorNota (Estructura e, nat nota)

Una vez que las materias son aprobadas (mediante una operación por fuera del alcance de este ejercicio) se quiere poder listar por pantalla las materias que hayan sido salvadas por la nota pasada por parámetro. Esta operación debe poder cumplirse en $O(1)$ peor caso (sin tener en cuenta el tiempo del listado).

Se pide:

1) **Diseño y definición en C++** de las estructuras de datos que permitan resolver las consultas anteriores sin duplicar información. **Justificando** las cotas temporales solicitadas.

2) **Codificación** de la operación 5

En caso de utilizar TADs auxiliares deberá especificarlos e implementar las operaciones que utilizó.

Duración: 3 horas

Con material: NO